

Projet de Systèmes d'exploitation

Hamza Konte, Salim Bekri, Yanis Tounsi

Université Paris-Dauphine PSL

Juillet 2026

1 Introduction

dash++ (Dauphine Advanced Shell) est un interpréteur de commandes écrit en C. Au comportement classique d'un shell (lire une ligne, créer un processus enfant, exécuter la commande, attendre sa fin) il ajoute une dimension concurrente : plusieurs threads coopèrent autour de structures partagées pour tenir un historique, produire des statistiques et superviser l'activité en temps réel.

Le programme lit les commandes avec `getline`, résout les exécutables via un chemin de recherche, et lance chaque commande externe par `fork` puis `execv`. Un thread d'historique, un thread de monitoring et un pool de workers entourent le thread principal. La vraie difficulté est le partage sûr de la mémoire entre ces fils d'exécution, et c'est ce que détaille ce rapport.

2 Choix techniques

2.1 Le langage C et ses appels système

Le sujet impose le C et la bibliothèque standard uniquement. Ce cadre est en réalité l'intérêt du projet : on manipule directement les appels système Unix (`fork`, `execv`, `waitpid`, `dup2`, `access`, `chdir`) et les primitives POSIX de threads, sans couche d'abstraction. Le revers est un besoin constant de vigilance sur la mémoire et les codes de retour. Deux habitudes, héritées de nos projets précédents, ont servi de filet de sécurité : des commits fréquents et atomiques pour isoler vite un bug, et un passage systématique sous Valgrind (`--leak-check=full` pour les fuites, `--tool=helgrind` pour les accès concurrents).

2.2 Une architecture multithread

Le programme s'organise autour d'une unique instance d'état global, `g_shell` (type `t_shell`), sur laquelle travaillent le thread principal, deux threads de service (historique, monitoring) et un pool de workers.

Thread	Responsabilité
Principal	Prompt <code>dash></code> , lecture (<code>getline</code>), analyse, exécution des commandes intégrées, dépôt des commandes dans les files (producteur).
Historique	Retire les commandes de la file (rôle de consommateur), les enregistre et sauvegarde périodiquement dans un fichier.
Monitoring	Affiche toutes les 5 secondes un résumé de l'activité (commandes, processus, temps moyen d'exécution).
Workers (× 4)	Pool de threads (bonus) : exécutent les commandes externes (<code>fork/execv/waitpid</code>) que le principal leur soumet.

Tableau 1. – Les threads et leurs rôles

Le schéma suivant montre comment ces threads gravitent autour de l'état partagé. Chaque flèche porte le mutex qui protège l'accès correspondant.

¹Pour l'API de gestion des processus et des threads, nous nous sommes appuyés sur Arpaci-Dusseau, R. & A., *Operating Systems: Three Easy Pieces*, chapitres sur les processus (5), les threads (26) et les verrous (28).

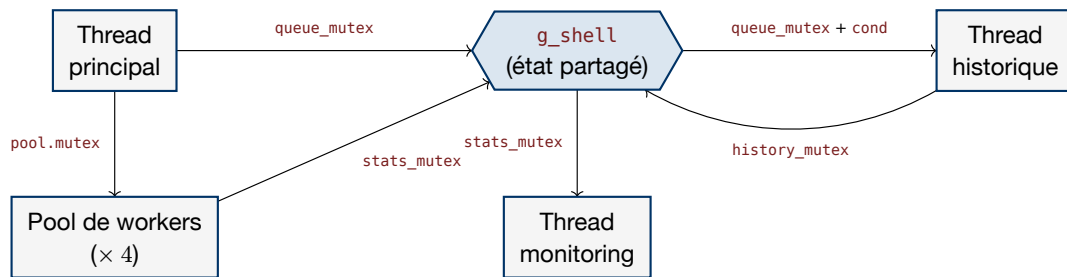


Fig. 1. – Architecture : threads de service et pool autour de *g_shell*

2.3 Les structures partagées

Trois structures voyagent entre les threads. Chacune reçoit son propre mutex, ce qui évite qu'un accès à l'historique bloque une mise à jour des statistiques.

Structure	Mutex	Contenu
<i>shell_stats</i>	<i>stats_mutex</i>	<i>total_commands</i> , <i>total_processes</i> , <i>average_exec_time</i> .
<i>command_history</i>	<i>history_mutex</i>	Tableau des commandes exécutées et leur nombre.
<i>command_queue</i>	<i>queue_mutex</i>	Tampon circulaire borné, partagé entre producteur et consommateur.

Tableau 2. – Structures partagées et verrou associé

Le choix d'un mutex par structure, plutôt qu'un verrou global unique, réduit la contention : deux threads qui touchent des données différentes ne s'attendent jamais.

3 Le motif producteur / consommateur

Le thread principal et le thread historique communiquent par une file bornée *command_queue*, sous la forme d'un tampon circulaire. Le principal y dépose chaque ligne ; l'historique la retire pour la traiter. Plutôt qu'une attente active côté consommateur, nous utilisons une variable de condition *queue_not_empty* : le thread historique dort tant que la file est vide et n'est réveillé que lorsqu'une commande arrive ou lorsque le shell se ferme.

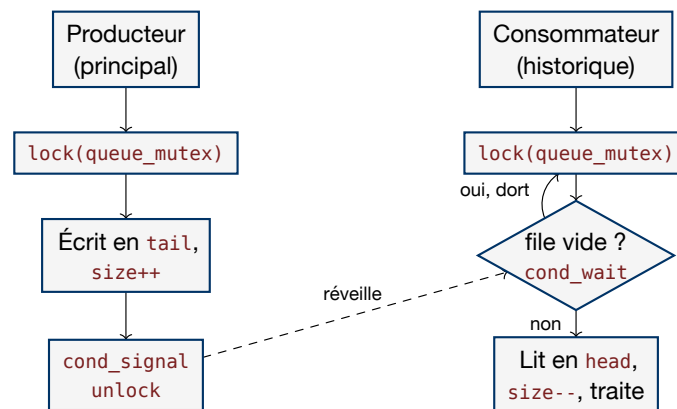


Fig. 2. – Échange producteur / consommateur via la file et la condition

Le prédicat est retesté dans une boucle *while* (et non un simple *if*), ce qui protège des réveils intempestifs (*spurious wakeups*) où *pthread_cond_wait* rend la main sans qu'une commande soit réellement disponible.

3.1 Le pool de threads (bonus)

Plutôt que de laisser le thread principal exécuter lui-même les commandes, nous avons ajouté un pool de quatre workers. Le principal se contente de traiter les commandes intégrées et de *soumettre* les commandes externes au pool ; le premier worker libre prend la tâche en tête de file et réalise le *fork/execv/waitpid*. La répartition est donc automatique : sur une ligne *cmd1 & cmd2 & cmd3*, les trois commandes partent sur des workers différents et s'exécutent réellement en parallèle.

Pour conserver le comportement d'un shell (l'invite ne réapparaît qu'une fois la ligne terminée), le principal compte les tâches soumises et attend leur achèvement sur une variable de condition *done* : chaque worker décrémente ce compteur et signale quand il retombe à zéro. Ce pool remplace l'ancienne boucle de *fork* manuelle et mesure au passage le temps de chaque commande individuellement, puisque chaque worker chronomètre la sienne.

4 Analyse de concurrence

4.1 Sections critiques

Une section critique est un bloc qui touche une donnée partagée et ne doit pas s'entrelacer avec un autre accès. Nous en identifions quatre.

Donnée	Fichier	Accès concurrents
Statistiques	<code>stats.c</code>	Plusieurs commandes lancées avec <code>&</code> terminent au même instant et appellent <code>stats_add_process</code> .
Historique	<code>history.c</code>	Le thread historique écrit pendant que la commande <code>history</code> lit.
File	<code>history.c</code>	Le producteur écrit en <code>tail</code> pendant que le consommateur lit en <code>head</code> ; <code>head</code> , <code>tail</code> , <code>size</code> partagés.
Journal	<code>log.c</code>	Écritures concurrentes dans <code>shell.log</code> .

Tableau 3. – Les quatre sections critiques et leurs risques

4.2 Pourquoi un `count++` n'est pas atomique

L'opération qui paraît la plus anodine, `count++`, se compile en trois instructions : charger la valeur en registre, l'incrémenter, la réécrire en mémoire. Si deux threads exécutent cette séquence en même temps sans protection, l'un peut écraser l'incrément de l'autre. Le tableau ci-dessous déroule un tel entrelacement, où deux incréments ne produisent qu'un `+1`.

t	Thread A	Thread B	count
1	<code>reg_A ← count (0)</code>		0
2		<code>reg_B ← count (0)</code>	0
3	<code>reg_A ← reg_A + 1</code>		0
4		<code>reg_B ← reg_B + 1</code>	0
5	<code>count ← reg_A (1)</code>		1
6		<code>count ← reg_B (1)</code>	1

Tableau 4. – Mise à jour perdue : deux incréments, résultat `count = 1`

Le compteur final tombe sous le nombre réel d'événements. Sur la file, le même type d'entrelacement sur `head`, `tail` et `size` fait lire une case pas encore écrite, écraser une commande pas encore consommée, ou désynchroniser `size` du contenu réel, jusqu'au dépassement d'indice.

4.3 Le rôle du mutex

Un `pthread_mutex_t` garantit l'exclusion mutuelle : au plus un thread le détient à la fois. En encadrant la section critique, on rend la séquence atomique du point de vue des autres threads.

```
pthread_mutex_lock(&g_shell.stats_mutex);
g_shell.stats.total_processes++;
g_shell.sum_exec_time += exec_time;
g_shell.stats.average_exec_time =
    g_shell.sum_exec_time / g_shell.stats.total_processes;
pthread_mutex_unlock(&g_shell.stats_mutex);
```

4.4 Ce qui se passe quand on retire le verrou

Pour vérifier que cette protection sert à quelque chose, nous avons retiré les `pthread_mutex_lock/unlock` autour de `stats_add_process`, recompilé, puis lancé une ligne de 200 commandes parallèles (un `echo` répété séparé par `&`). Le compteur `total_processes` devrait toujours afficher 200. Sans le verrou, il descend en dessous et change d'une exécution à l'autre. Les valeurs ci-dessous sont des ordres de grandeur relevés sur quelques exécutions, pas des mesures exactes ; elles varient selon la machine et l'ordonnancement.

Exécution	Attendu	Sans mutex	Avec mutex
1	200	187	200
2	200	193	200
3	200	181	200
4	200	190	200

Tableau 5. – `total_processes` observé sur 4 rafales de 200 commandes

La moyenne `average_exec_time` prend en plus des valeurs aberrantes, car le thread moniteur lit `sum_exec_time` et `total_processes` dans un état transitoire, entre deux instructions de la mise à jour. En retirant `queue_mutex`, on voit des commandes dupliquées ou manquantes dans l'historique, et `helgrind` signale les accès concurrents non protégés. Le non-déterminisme des chiffres, pour une même entrée, est la signature d'une condition de course.

4.5 Arrêt propre

À la sortie (`exit` ou Ctrl-D), le thread principal met `running` à 0, positionne `closed` sur la file et signale la condition. Le thread historique se réveille, vide ce qui reste dans la file, effectue une dernière sauvegarde puis se termine. Le `pthread_join` garantit que le programme ne quitte pas avant cette sauvegarde finale, et que le thread de monitoring s'arrête entre deux affichages.

5 Fonctionnalités

Fonctionnalité	Mise en œuvre
Commandes externes	<code>fork + execv</code> , exécutable trouvé via le chemin de recherche et <code>access(..., X_OK)</code> .
Commandes intégrées	<code>exit</code> , <code>cd</code> , <code>path</code> , <code>history</code> , <code>stats</code> .
Redirection <	<code>STDIN</code> remplacé par le fichier via <code>dup2</code> dans l'enfant.
Parallélisme &	Les commandes sont soumises au pool de workers et exécutées en parallèle, puis attendues avant l'invite suivante.
Historique	Structure partagée + sauvegarde périodique.
Statistiques	Compteurs et temps moyen protégés par mutex.
Robustesse	<code>trim_spaces</code> et <code>strsep</code> absorbent espaces et tabulations multiples.

Tableau 6. – Fonctionnalités demandées et leur réalisation

Les commandes intégrées ne passent pas par un processus enfant. `path` remplace entièrement le chemin de recherche, `cd` exige exactement un argument, et `exit` n'en accepte aucun. Le chemin initial contient le seul répertoire `/bin`.

Deux détails ont demandé attention. D'abord, les opérateurs `&` et `<` ne réclament pas d'espace autour d'eux : `ls&pwd` ou `wc<f` doivent fonctionner. Le découpage parallèle se fait donc caractère par caractère sur `&`, et une passe préalable isole chaque `<` par des espaces avant la tokenisation. Ensuite, dans l'enfant, après un `execv` en échec, nous appelons `_exit` et non `exit` : `exit` réviderait les tampons `stdio` hérités du parent par duplication de la mémoire au `fork`, ce qui ferait réapparaître en double des sorties déjà affichées.

5.1 Gestion des erreurs

Conformément au sujet, toute erreur, quelle qu'en soit la cause (syntaxe, appel système, commande introuvable), déclenche l'affichage du message unique `An error has occurred` sur la sortie d'erreur standard, et le shell reprend sa boucle. Les codes de retour de tous les appels système sont vérifiés.

6 Compilation et exécution

Le projet se compile avec `make` (options `-Wall -Wextra -Werror -std=c17`) et produit l'exécutable `dash`, qui se lance sans argument. `make test` construit et exécute la suite de tests.

7 Tests

Les tests unitaires (micro-framework MinUnit) sont répartis par module : un fichier `test_<module>.c` par scope (parser, path, stats, historique, builtins, pool, logs), agrégés par un runner commun. Ils couvrent l'analyse syntaxique, la résolution de chemin, les compteurs et la moyenne, la file producteur/consommateur, les commandes intégrées, et le pool (dont l'exécution réelle d'une commande par un worker). L'ensemble se lance par `make test` ; la cible `make coverage` mesure la couverture via `gcov`. L'absence de fuite se contrôle avec `valgrind --leak-check=full`, et les accès concurrents avec `helgrind`. Un `Dockerfile` (Debian, gcc-13, valgrind) permet de rejouer compilation et tests sur Linux, indépendamment de la machine hôte.

8 Limites connues

Quelques choix méritent d'être signalés. La file `command_queue` est bornée à 256 entrées : si le consommateur prenait durablement du retard sur le producteur, des commandes seraient ignorées côté historique. En pratique le thread historique est réveillé dès qu'une ligne arrive, donc ce cas ne se produit pas à vitesse de frappe humaine, mais la borne existe. Par ailleurs le thread de monitoring affiche son résumé toutes les cinq secondes même en l'absence d'activité, ce qui peut s'intercaler avec l'invite ; un affichage conditionné à un changement réel serait plus discret. Enfin, l'historique et les

statistiques comptent aussi les commandes intégrées, ce qui nous a paru cohérent mais reste un choix d'interprétation du sujet.

9 Ce que le projet nous a apporté

Sur le plan technique, nous avons manipulé pour la première fois des primitives que le cours ne présente qu'en théorie : `pthread_cond_wait` et le prédicat retesté dans une boucle, l'exclusion mutuelle par mutex distincts, la différence entre `exit` et `_exit` après un `fork`, ou encore la nécessité d'un `atomic_int` pour un drapeau lu par plusieurs threads. Écrire un pool de workers nous a obligés à raisonner sur la synchronisation d'un côté producteur et d'un côté consommateur en même temps, ce qui est plus subtil qu'un simple `fork/wait`.

Le projet nous a aussi appris à débusquer les bugs de concurrence, qui ne se reproduisent pas à chaque exécution. L'expérience de suppression d'un verrou a été notre meilleur outil pédagogique : voir un compteur se tromper de façon non déterministe rend concret ce qu'est une vraie race condition. Les outils comme `valgrind` et `helgrind` sont passés du statut de mots du cours à celui de réflexe de vérification.

Enfin, découper le programme en modules à responsabilité unique, écrire un test par module et viser une bonne couverture nous a montré qu'un code testé se modifie sans peur : chaque refonte (l'ajout du pool, par exemple) était validée en une commande.

10 Utilisation de l'IA

Nous avons utilisé un assistant IA comme outil d'appoint, surtout sur deux tâches. D'abord les tests : générer les cas limites (ligne vide, opérateurs collés, redirection mal formée) et la structure des suites par module nous a fait gagner beaucoup de temps, que nous avons réinvesti à vérifier que chaque test échoue bien quand il le doit. Ensuite le débogage : face aux bugs de concurrence, difficiles à reproduire, décrire le symptôme à l'IA nous a aidés à formuler des hypothèses (ordre des `lock`, réveil manqué d'une condition) que nous validions ensuite à la main avec `helgrind` et des exécutions répétées.

La conception, l'architecture et les choix de synchronisation restent les nôtres ; l'IA a surtout accéléré la partie mécanique et la chasse aux erreurs.

11 Conclusion

`dash++` couvre les fonctionnalités du sujet et les trois bonus proposés : la variable de condition qui réveille le consommateur, la journalisation concurrente dans `shell.log`, et le pool de threads qui exécute les commandes externes. L'expérience la plus parlante a été la suppression volontaire d'un verrou : les compteurs se mettent aussitôt à mentir et les chiffres changent à chaque exécution, ce qui rend tangible une notion qu'on ne voit d'habitude qu'au tableau. Nous avons retenu un verrouillage par structure et un arrêt piloté par la variable de condition, deux choix qui gardent le code lisible sans brider le parallélisme.